Öğrenci 2 ne yaptı? 2

Doğrulama sonuçları ve savunma notları. Öğrenci 1, Öğrenci 3 ve sunum hazırlığı için.

#32
Temiz klonda Optuna yine aynı en iyi denemeyi buldu

5 / 5
Yeniden üretilen çıktı dosyası commit'li halle birebir aynı

0,6986 · 0,6169
LightGBM ve Perceptron test Macro-F1, her yerde aynı

İlk sunum (`Öğrenci 2 ne yaptı.pdf`) neyin yapıldığını anlatıyor. Bu sunum, o işin tekrarlanabildiğini ve hocanın sorabileceği parametre sorularını kapsıyor.

Bana verilen 3 iş: üçü de tamam

Ekip listesindeki sırayla.

İş	Sonuç	Kanıt
Temiz klonda 03'ü baştan sona çalıştır, Optuna yine #32'yi buluyor mu?	Evet	#32, CV PR-AUC 0,5667 ± 0,0125, 176 ağaç. Hata yok.
Kendi "ne yaptım" sunumunu GitHub'a koy	Tamam	Commit `a3b1673`, `ArmaganBranch`
PR #1 ve #3'te final sonuçlar (0,6986 / 0,6169) aynı mı?	Aynı	Tahmin dosyaları bit bit aynı. README, `final_metrics.csv` ve 05 aynı sayıyı yazıyor.

> Ek olarak bu sunum: önceki sunumda açık kalan iki soruyu kapatıyor (2000 ağaç tavanı ve Perceptron epoch aralığı), ve hocanın sorabileceği parametre sorularına kısa cevaplar veriyor.

Temiz klon: Optuna yine #32'yi buldu

GitHub'dan boş klasöre klonlandı. Optuna veritabanı yoktu, yani 50 deneme sıfırdan çalıştı.

Sıra	Deneme	CV PR-AUC
1	**#32**	**0,5667**
2	#48	0,5663
3	#44	0,5661
4	#37	0,5658
5	#43	0,5657

#32'nin değeri commit'li JSON'daki değerle 6 haneye kadar aynı: 0,566735.

Kontrol	Sonuç
Biten / kesilen deneme	20 / 30 (ilk sunumdaki gibi)
Süre	Toplam yaklaşık 8 dk. Optuna 4,1–4,5 sn/deneme.
Defter hatası	Yok, çıkış kodu 0
Uyarı ("arama başka denemeyi buldu")	Basılmadı

> **Varsayım:** Kod ve veri temizdi, ama paketler sıfırdan kurulmadı. Bilgisayardaki `.venv` kullanıldı, yani kütüphane sürümleri aynıydı. İlk sunumdaki 13 → 9 sn/deneme ile buradaki 4,5 sn arasındaki farkın nedenini ölçmedim.

Yeniden üretilen çıktılar birebir aynı

Defter 5 dosyayı yeniden yazdı. Commit'li halleriyle içerik olarak karşılaştırıldı.

Dosya	Ne içeriyor	Sonuç
`models/preds/lgbm.csv`	LightGBM test tahminleri (4495 satır)	Aynı
`models/preds/perceptron.csv`	Perceptron test tahminleri	Aynı
`models/importance/lgbm_importance.csv`	gain ve split önem tablosu	Aynı
`reports/perceptron_sweep.csv`	50 ayar × 5 tohum taraması	Aynı
`reports/train_variant_comparison.csv`	clean / SMOTE / CTGAN kıyası	Aynı

> Git bu dosyaları "değişti" diye gösterdi. Tek fark Windows satır sonu (CRLF). Satır sonları yok sayılınca fark sıfır.

Anlamı: Perceptron taraması, LightGBM final modeli ve tahminler tohumlarla tam sabitlenmiş. Başka biri aynı ortamla çalıştırırsa aynı sayıları alır.

Final sonuçlar her yerde aynı

Önce sayılar tahmin dosyalarından yeniden hesaplandı, sonra her kaynakla kıyaslandı.

Nerede	LightGBM	Perceptron	Durum
Tahmin dosyalarından yeniden hesap	0,6986	0,6169	Referans
PR #2 açıklaması (Ö2)	0,6986	0,6169	**Aynı**
Ö3 dalı: `models/preds/*.csv`	—	—	**Bit bit aynı**
Ö3 dalı: `reports/final_metrics.csv`	0,69860	0,61690	**Aynı**
Ö3 dalı: `README.md`	0,699	0,617	**Yuvarlanmış**
Ö3 dalı: `05_evaluation_plots`	0,698600	0,616902	**Aynı**
PR #1 (Ö1)	—	—	Geçmiyor, veri tarafı

Yan metrikler de tutuyor. LightGBM / Perceptron: ROC-AUC 0,772 / 0,647 · PR-AUC 0,552 / 0,334 · weighted-F1 0,795 / 0,740 · accuracy 0,799 / 0,745.

`04_tabnet` 'teki sayılar TabNet'e ait, kıyasa katılmadı.

GitHub durumu

Ö2'nin bütün commit'leri GitHub'da. Ama hiçbiri henüz `main`'de değil.

PR	Dal	Durum
#1 Ö1	`ogrenci1/veri`	Açık, birleşmedi
#2 Ö2	`ArmaganBranch`	Açık, birleşmedi
#3 Ö3	`o3/kadirgonullu`	Açık, birleşmedi

`main`'de yalnızca ilk ortak iskelet (`afef5bd`) var. Birleştirme sırası: Ö1 → Ö2 → Ö3. PR'larda yorum yok.

Bilerek gönderilmeyen	Neden
`models/optuna_lgbm.db`	Yeniden üretilir. Sonuç JSON'da sabit.
`data/raw/`	İlk çalıştırmada indirilir
`.venv/`, önbellekler	Ortam dosyaları
Ö1 ve Ö3'ün "ne yaptı" PDF'leri	Sahipleri kendisi gönderir

> 05 defteri tahmin dosyalarını okuyor. Ö2 PR'ı `main`'e girmeden bu dosyalar `main`'de olmayacak.

Perceptron parametreleri ne demek?

Parametre	Anlamı
`eta0` (η)	Öğrenme oranı: her hatada adım büyüklüğü
`penalty`	Ceza türü: yok, `l2`, `elasticnet`
`alpha` (α)	Cezanın gücü. `penalty` yoksa etkisiz.
`class_weight`	`balanced`: azınlığa ≈2,26, çoğunluğa ≈0,64 ağırlık (oran 1:3,52)
`max_iter`	En fazla epoch: 1000
`random_state`	Örneklerin karıştırılma sırası

```
skor = w·x + b          # y ∈ {-1, +1}
y·skor ≤ 0 ise (hata):
  w ← w + η · ağırlık · y · x
  b ← b + η · ağırlık · y
L2 varsa her örnekte:
  w ← (1 − η·α) · w
```

`eta0` tek başına neden etkisiz? w sıfırdan başlar, ceza yokken w = η × (bir toplam). η hepsini aynı oranda ölçekler, tahminin işareti değişmez. Fark yalnızca ceza terimi (η·α) ve mutlak `tol` 'den gelir.

Basitleştirme: L1 ve elasticnet'in kırpma güncellemesi gösterilmedi.

Perceptron: katman, tavan, durdurma, tarama

Soru	Cevap
Kaç katman?	1. Gizli katman yok, girdiler tek çıktıya bağlı.
Tavan	`max_iter=1000` epoch (LightGBM'deki 2000'in karşılığı)
Pratik taban	6 epoch: ilk epoch referans, sonra 5 epoch sabır
Gerçekte kaç epoch?	50 ayarın hepsinde 6–30,2 (5 tohum ort.). Seçilen ayarda 7.
Durdurma ölçütü	`tol=1e-3`, `n_iter_no_change=5` (varsayılan)
`early_stopping`	**Kapalı.** Eğitim kaybına bakılıyor, val parçası ayrılmıyor.

Parametre	Denenen	Seçilen
`eta0`	1e-4 … 1,0 (5 değer)	**0,001**
`penalty`	yok, l2, elasticnet	**l2**
`alpha`	1e-4, 1e-3	**0,001**
`class_weight`	yok, balanced	**yok**

50 kombinasyon × 5 tohum = 250 model. Seçilen: val macro-F1 0,636 ± 0,030.

> Mutlak `tol` yüzünden küçük `eta0` erken durur: `eta0=1e-4` satırlarının hepsi 6 epoch.

LightGBM'de dal budama yaptık mı?

Hayır. "Büyüt, işe yaramadıysa geri dönüp kes" (post-pruning) yok. Dal baştan açılmıyor (pre-pruning).

Dalı sınırlayan	#32	Etkisi
`max_depth`	9	En fazla 9 seviye
`num_leaves`	66	Ağaç başına en fazla 66 yaprak
`min_child_samples`	14	14'ten küçük yaprak açılmaz
`reg_alpha` (L1)	0,76	Küçük kazançlı bölmeyi eler
`reg_lambda` (L2)	0,07	Kazancı ve yaprak değerini küçültür

`min_split_gain` verilmedi (varsayılan 0): yalnızca pozitif kazançlı bölme yapılır.

Karışabilecek üç "kesme"

- **Early stopping:** yeni *ağaç* eklemeyi durdurur (#32'de 176 ağaç).
- **MedianPruner:** kötü giden *Optuna denemesini* keser (50'de 30).
- **subsample / colsample:** satır ve kolonları *rastgele* seçer (≈%58 / ≈%73). Faydasız veri seçilip atılmaz.

> Hiçbir aşamada var olan bir dal geri dönülüp kesilmiyor.

Neden 2000? Sabit değerler nereden geliyor?

Sayıların çoğu hesaplanmış "doğru" değer değil. Ya "yeterince büyük" tavan, ya yaygın pratik.

Değer	Neden
`n_estimators=2000`	Hedef değil, tavan. Gerçek sayıyı early stopping seçiyor. **Yeni kontrol:** tamamlanan 20 denemede en çok 257 ağaç, medyan 190. Tavana yaklaşan yok. Final modele 2000 değil 176 veriliyor.
`EARLY_STOP=100`	PR-AUC turdan tura oynar. Kısa sabır geçici düşüşte keser.
`N_FOLDS=5`	Süre ile skor güvenilirliği arasında yaygın denge
`N_TRIALS=50`	Bütçe tercihi. Önce 10 denemeyle süre ölçülür, sonra 50'ye tamamlanır.
`SEED=42`	Anlamı yok, tekrarlanabilirlik için
`metric=average_precision`	Dengesiz veride ROC-AUC'den dürüst, eşikten bağımsız
`subsample_freq=1`	Verilmezse LightGBM `subsample`'ı yok sayar
`n_startup_trials=5`, `n_warmup_steps=1`	Pruner ilk 5 denemeyi ve ilk fold'u kesmez: kıyas için önce veri birikmeli

Kesilen 30 denemenin ağaç sayısı kaydedilmiyor. "Tavana yaklaşan yok" yalnızca tamamlanan 20 deneme için.

Arama aralıkları ve tuzak sorular

Parametre	Aralık	#32
`num_leaves`	15–255 (log)	66
`learning_rate`	0,01–0,2 (log)	0,017
`max_depth`	3–12 veya sınırsız	9
`min_child_samples`	10–200	14
`subsample`	0,5–1,0	0,58
`colsample_bytree`	0,5–1,0	0,73
`reg_alpha`, `reg_lambda`	1e-8–10 (log)	0,76 · 0,07
`scale_pos_weight`	1–4	1,006

Aralıklar ekip planından (bölüm 8, adım 3). Varsayılanları (31, 0,1, 20) içine alıyorlar.

- **Neden log ölçek?** Önemli olan oransal değişim: 0,01 → 0,02 iki kat, 0,19 → 0,20 neredeyse aynı.
- **Neden 0 değil 1e-8?** Log ölçekte 0 tanımsız. 1e-8 pratikte "ceza yok".
- **max_depth 9 ile num_leaves 66 çelişir mi?** Hayır. $2^9 = 512$ yaprak mümkün, asıl sınırı 66 koyuyor.
- **scale_pos_weight neden 4'e kadar?** Sınıf oranı 3,52. 4 tam dengeyi de kapsıyor.
- **Neden ≈1 çıktı?** Ağırlık PR-AUC'yi belirgin artırmamış. Eşik zaten val'de seçiliyor. Yorum, kanıt değil.
- **CV skoru tam dürüst mü?** Hafif iyimser: aynı fold hem durdurma hem skor için. Tüm denemeler aynı ölçüldüğü için kıyas adil.

Optuna ne?

Hiperparametre arama kütüphanesi. Modeli kendisi eğitmez, LightGBM'in ayarlarını arar.

Bir deneme (trial)

- `suggest_lgbm` aralıklardan bir set seçer
- LightGBM 5-fold CV'de eğitilir
- PR-AUC ortalaması skor olarak döner
- Optuna kaydeder, sıradakini buna göre seçer

> **Neden grid değil?** 9 parametre × 5 değer = $5^9 \approx 2$ milyon kombinasyon, her biri 5 fold. Perceptron'da 50 kombinasyon olduğu için grid yetti.

Parça	Görevi
TPE (sampler)	"Sırada ne denensin?" İlk 10 rastgele. Sonra denemeleri iyi/kötü ayırır, iyilerde sık görülen değerlere yönelir.
MedianPruner	"Devam edilsin mi?" Her fold'dan sonra önceki denemelerin medyanıyla kıyaslar, altındaysa keser.
sqlite kaydı	`optuna_lgbm.db`. Kesilirse kaldığı yerden devam eder.

Pruner'a skorların toplamı raporlanıyor. Ortalama raporlanınca hiçbir deneme kesilmiyordu (ilk sunum, sayfa 11).

split payı, split sırası, gain payı

- **split:** özellik tüm ağaçlarda kaç bölmede kullanıldı
- **split payı:** bu sayının toplam bölmeye oranı
- **split sırası:** bölme sayısına göre sıra, 1 en sık
- **gain payı:** o bölmelerin kaybı ne kadar düşürdüğünün toplamdaki payı

> **İki farklı "bölme":** ağaçtaki bölme modelin öğrenme yolu. Toplama bölme ise iki farklı ölçeği kıyaslanabilir yapmak için, yani yorum amaçlı.

Özellik	Gain payı	Split payı	Split sırası
`PAY_0`	%32,9	%2,6	15
`PAY_2`	%8,8	%1,5	18
`BILL_AMT1`	%5,2	%8,4	1
`LIMIT_BAL`	%4,9	%7,2	2

PAY_0 az kullanılıyor ama çok kazandırıyor. En sık bölünen en önemli değil.

Olası neden: sürekli kolonlarda çok eşik var, küçük kazançla tekrar tekrar kullanılıyor. Test edilmedi. Önem neden-sonuç göstermez.

Sentetik veri yaradı mı? Skor nasıl şişerdi?

Yaramadı: iki modelde de en iyi `clean`. Şişme bizde olmadı, çünkü yalnızca gerçek val'de ölçtük.

Model	Train	Macro-F1	PR-AUC
LightGBM	**clean**	**0,709**	**0,550**
LightGBM	SMOTE	0,695	0,534
LightGBM	CTGAN	0,689	0,527
Perceptron	**clean**	**0,636**	**0,407**
Perceptron	SMOTE	0,567	0,288
Perceptron	CTGAN	0,584	0,310

Gerçek val, 5 tohum ortalaması. Neden yaramadı: eşik zaten val'de seçiliyor, sentetik satırlar gerçek dağılımdan sapıyor.

```
A (gerçek):   LIMIT_BAL = 50.000
B (gerçek):   LIMIT_BAL = 60.000
S (SMOTE):    LIMIT_BAL = 55.000
```

Artırılmış veride CV yapılsa S eğitimde, A ölçüm fold'unda kalabilir. Model A'nın neredeyse kopyasını görmüş olur. Bu bir **sızıntı**.

İkinci etki: ölçüm fold'u da 1:1 olur. Rastgele PR-AUC 0,22 yerine 0,50 olur, skor gerçek dağılımla kıyaslanamaz.

> CV şart olsaydı: SMOTE her fold'da yalnızca o fold'un eğitim parçasına uygulanırdı.

Neden 5 tohum?

5'in özel anlamı yok. Sonuç tek bir tohumun şansına bağlı kalmasın, maliyet de makul olsun diye.

Model	Tohumlar arası std	Anlamı
Perceptron (seçilen)	≈0,030	Tek tohumla kıyas yanıltıcı
LightGBM	≈0,001	Tohum sonucu pek değiştirmiyor

Tohum	Ortalamanın std hatası
1	0,030
5	≈0,013
20	≈0,007

`std / √n`. Tohumları bağımsız sayan bir basitleştirme.

- **Tohum neyi değiştirir?** Perceptron'da örnek sırası, LightGBM'de satır/kolon örneklemesi. Veri bölmesi `splits.csv`'de sabit.
- **Neden 20 değil?** 5 hatayı yarıdan fazla düşürüyor. 20 hatayı ancak yarıya indirir, maliyeti 4 katına çıkarır.
- **5 yeterince kesin mi?** Perceptron'da tam değil: seçilen 0,636 ile bir sonraki 0,631 arası fark bir std hatasından küçük.
- **Final de 5 tohumla mı?** Hayır. 5 tohum seçim için. Final Perceptron tek tohum (`SEED=42`): val 0,594, test 0,617.

Bilinen sınırlar ve sonraki adımlar

Bilinen sınırlar

- LightGBM CV skoru hafif iyimser: aynı fold durdurma ve skor için kullanılıyor.
- Val skorları da hafif iyimser: eşik aynı val'de seçiliyor. Nihai sayı test.
- Perceptron'da en iyi birkaç ayar gürültü içinde. "En iyisi kesin bu" denemez.
- Temiz klon kontrolü aynı `.venv` ile yapıldı. Sıfır kurulumla denenmedi.

Sonraki adımlar

Adım	Kim
PR #1 (`ogrenci1/veri`) → `main`	Ö1
PR #2 (`ArmaganBranch`) → `main`	Ö2, sıra gelince
PR #3 (`o3/kadirgonullu`) → `main`	Ö3
Temiz klonda 00 → 05 kontrol	Herkes
Restart & Run All, `nbqa flake8`	Ö2